

PRÁCTICA

1) (10ptos) Indique las acciones que realizan las siguientes secuencias de comandos:

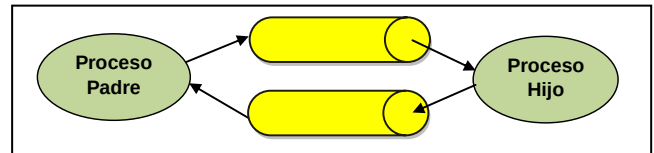
a) `ps ax | sort > salida` en GNU/Linux

b) `ATTRIB | more` en Windows

2) (15ptos) Arme el árbol de procesos generado por el siguiente código

```
int i, j ,k;
for(i=0;i<2;i++){
    if (fork()==0){
        if (i==1) {
            for(j=0;j<2;j++){
                if (fork()==0){
                    if (j==1){
                        for(k=0;k<2;k++){
                            if (k==1) {
                                if (fork()!=0) break;
                            }
                        }
                    }
                }
            }
        }
    }
}
```

3) (15ptos) Se desea realizar la comunicación bidireccional entre procesos, que permita a un proceso padre enviar datos al proceso hijo y que luego el proceso hijo enviar datos a un proceso padre. Describa la operatoria y consideraciones para resolverlo mediante tubería (pipe).



4) (25ptos) Considerando el siguiente programa que produce valores y calcula e imprime su doble: Modifíquelo para que, mediante la sincronización con semáforos, permita controlar la secuencia y el acceso a sección crítica.

```
1  const int num_iter = 100; // Numero de iteraciones
2  const int tam_vector = 5; // Tamaño del vector
3
4  int vector[tam_vector]; // Vector para producir o consumir
5  int pos_productor = 0;
6  int pos_consumidor = 0;
7  int doble;
8
9  int producir_dato() {
10     static int dato = 0;
11     dato++;
12     return dato;
13 }
14
15 void duplicar_dato( int dato ) {
16     doble = dato*2;
17     cout << "Dato duplicado: " << doble;
18 }
19
20 void * productor( void * p ) {
21     for( unsigned long i = 0 ; i < num_iter ; i++ ){
22         int dato = producir_dato(); // Produce un dato
23         vector[pos_productor%tam_vector] = dato;
24         pos_productor++;
25         cout << "Dato producido: " << dato;
26     }
27     return NULL ;
28 }
```

```

30 void * consumidor( void * p ) {
31     for( unsigned long i = 0 ; i < num_iter ; i++ ){
32         int dato = vector[pos_consumidor*tam_vector];    // Consume el valor gene
33         pos_consumidor++;
34         duplicar_dato(dato);
35     }
36     return NULL ;
37 }
38
39 int main(int argc, char **argv) {
40     pthread_t h1,h2;
41     pthread_create(&h1,NULL,productor,NULL);
42     pthread_create(&h2,NULL,consumidor,NULL);
43     pthread_join(h1,NULL);
44     pthread_join(h2,NULL);
45     return 0;
46 }

```

5) (35ptos) Realice los diagramas de Gantt para planificaciones **FCFS**, **RR Q=3**, **SRTF**, **prioridad no apropiativa**, para el cuadro de procesos de la derecha:

Obtener además el tiempo medio de espera de cada proceso en cada planificación.

Proceso	Ráfaga de CPU	Tiempo de llegada	Prioridad
A	5	2	2
B	3	0	3
C	4	0	1
D	7	3	3
E	6	3	2

TEORÍA

- (20ptos) Esquematice y mencione las ventajas y desventajas de las arquitecturas monolíticas y microkernel.
- (15ptos) En qué se diferencia una system call de una llamada a una función común. Esquematice la system call.
- (20ptos) Describa brevemente cada estado de un proceso. Explique si un proceso puede evolucionar o no del estado bloqueado al estado de ejecución.
- (16ptos) Mencione al menos 4 objetivos de la planificación de procesos.
- (11ptos) De qué manera un algoritmo por prioridades, puede evitar la inanición de procesos?
- (18ptos) Responda **verdadero** o **falso**
 - El hardware proporciona diferentes niveles de privilegios mediante un modo dual de operación: modo usuario y modo kernel
 - Cuando se bloquea un hilo a nivel de kernel, se bloquean todos los hilos del proceso.
 - En el modo de comunicación asíncrono, cuando se envía un mensaje, el emisor se bloquea hasta que se recibe el mismo.
 - Cuando se envía un mensaje mediante una comunicación indirecta, se realiza pasando como parámetro el proceso destino y el mensaje.
 - Para comunicar información entre 2 hilos, únicamente se puede hacer con tuberías sin nombre.
 - La condición de carrera es la situación donde el resultado obtenido depende del orden de ejecución de los procesos.
 - El mutex es un semáforo binario
 - El SRTF es un algoritmo de planificación apropiativo, ya que cuando un proceso comienza a ejecutarse en la CPU, no sale hasta que termina.
 - El algoritmo Round Robin ofrece mejores tiempos de respuesta que el FCFS porque ejecuta los procesos más cortos primero.